第 2 章 • 预备知识 / 2.2

2.2 数据预处理

从含有缺失值与文字的原始表格，得到模型可计算的张量。

读取表格 → 处理缺失 → 编码类别 → 转换与核对

本节目标

读完后，能够分开特征与标签，解释填补或删除的选择，将类别特征编码成数值，并按同一套规则转换成无缺失的张量。

建议先连续阅读完整知识链，再运行示例，最后集中完成章末练习。已会的内容可快速通过，重点追踪自己无法解释的结果。

教材定位：PDF 第 65-68 页 / 正文第 47-50 页。

参考《动手学深度学习》Release 2.0.0（2023-08-18）及本项目已验证讲义整理。此 PDF 为独立学习读本，配套程序与练习均在项目内。

为什么需要预处理

上一节的数据已经是整齐的数值张量。真实输入常包含文件、字符串、缺失项和不同数据类型，需要先转换成模型能计算的数值结构。

DataFrame 是 pandas 的表格对象，有行、列名和各列的数据类型。CSV 用逗号分隔字段，一行是一条记录。

沿用教材四条房屋样本：

NumRooms	Alley	Price
缺失	Pave	127500
2	缺失	106000
4	缺失	178100
缺失	缺失	140000

NumRooms 是房间数，Alley 是巷子类型，Price 是房价标签。这是教学数据，不代表实际市场。

2.2.1 读取并分开特征与标签

```python
import pandas as pd

data_file = "datasets/ch02/house_tiny.csv"
data = pd.read_csv(data_file)
inputs = data.iloc[:, :2].copy()
targets = data.iloc[:, 2].copy()
```

read_csv 根据表头建立列名，识别数值和默认的缺失标记。CSV 中的 NA 在本例被读取为 NaN。

iloc 按位置取数据：第一项选行，第二项选列；冒号表示全部，切片不包含终点。前两列是输入，第三列是待预测的真实标签。copy 创建独立表格或序列，后面的赋值不修改原始表。

按名字选列也可以：`data['Price']` 取标签，`data.drop(columns=['Price'])` 取其他列作为输入。

观察缺失：

```python
data.isna()       # 每个位置是否缺失
data.isna().sum() # DataFrame 默认按列汇总，得到各列缺失数量
```

这里 pandas DataFrame.sum 默认按列；上一节 torch.Tensor.sum 默认对全部元素求和，不要混淆不同库的默认行为。判断缺失用 isna，不用与 NaN 做相等比较。

2.2.2 数值缺失：填补与删除

缺失代表没有记录，不能自动理解为 0。房间数量未知不等于房间数量为零。

教材采用已知数值的均值作为替代：

```python
room_mean = inputs['NumRooms'].mean()
inputs['NumRooms'] = inputs['NumRooms'].fillna(room_mean)
```

mean 默认忽略缺失项，因此只使用 2 和 4，均值为 3。填补后房间数为 `[3,2,4,3]`。

均值填补假设已有记录的平均值可作为未知值的基准替代。它不是恢复了真实房间数，可能偏离某个样本的真实值；中位数、其他固定值或专门模型也是可能的方案，需结合任务选择。

处理多个数值列时，可以按各自均值填补：

```python
numeric_means = inputs.mean(numeric_only=True)
filled = inputs.fillna(numeric_means)
```

numeric_only=True 只汇总数值列；fillna 按列名对齐这些均值，类别缺失暂时不动。不要照搬旧版教材对混合类型整个表直接 mean 的写法，在现有 pandas 2.0.3 中可能尝试对字符串计算均值。

另一条路线是删除：

```python
data.dropna()                        # 删除任一列缺失的整行
data.dropna(subset=['NumRooms'])     # 只根据房间数缺失筛选行
inputs.drop(columns=['Alley'])       # 删除指定列
```

本例四行都含有至少一项缺失，直接 dropna 会得到空表。删除行会减少样本，删除列会减少特征；填补则保留它们但引入替代值假设。

教材的扩展练习要求删除缺失最多的列，可计算：

```python
missing_counts = inputs.isna().sum()
column_name = missing_counts.idxmax()
reduced = inputs.drop(columns=[column_name])
```

idxmax 返回最大值所在的索引标签，在这个按列汇总的结果中就是列名。本练习有唯一的最大缺失列。真实任务选择删列还需结合特征作用，不由缺失数量单独决定。

如果删行，特征和标签必须按相同的行一起筛选，保持一条样本对应一个正确标签。本例主流程只填补和编码，不删行。

类别特征：独热编码

Alley 中 Pave 是文字。对无序类别直接指定 0、1、2，并当作数值输入线性模型，会引入人为大小和距离关系，例如认为 2 比 1 大、1 位于 0 与 2 中间。这些关系未必符合类别本身。

独热编码为不同状态建立不同列：

```python
encoded = pd.get_dummies(
    inputs,
    columns=['Alley'],
    dummy_na=True,
    dtype=np.float32,
)
```

dummy_na=True 将缺失也编码为一个状态；它表示记录缺失，不自动等于“没有巷子”。

Alley 原值	Alley_Pave	Alley_nan
Pave	1	0
缺失	0	1

最终的特征列是 NumRooms、Alley_Pave、Alley_nan，共三个数值特征。指定 float32 让编码列直接成为浮点数。

get_dummies 也可以不指定 columns，默认编码字符串等类别列。综合练习在删除 Garage 后，对剩余 Alley 的 Pave、Gravel、缺失三个状态编码。

2.2.3 转换成张量

先将整表统一转换成数值 NumPy 数组，再创建张量：

PYTHON

```
X = torch.tensor(encoded.to_numpy(dtype=np.float32), dtype=torch.float32)
y = torch.tensor(targets.to_numpy(dtype=np.float32), dtype=torch.float32)
y = y.reshape(-1, 1)
```

本例结果：

结果 / 结构

```
X = [[3,1,0],
     [2,0,1],
     [4,0,1],
     [3,0,1]]

y = [[127500],
     [106000],
     [178100],
     [140000]]
```

X.shape=(4,3)：四条样本，每条三个特征。y.shape=(4,1)：四条样本，每条一个房价。按这一约定，后续模型输出也用 (N,1)，避免上一节的意外广播。

书中使用 float64 和一维标签，本项目显式使用 float32 与列形式标签，方便衔接常见 PyTorch 模型；数据值与书中相同。

转换成张量不是清洗本身：NaN 可以进入浮点张量并影响后续计算。因此程序在转换前检查有限数值，转换后打印形状、类型和是否仍有 NaN。

程序将清洗后的特征表导出到 `reports/ch02/2_2_processed_features.csv`，并保持原样本行顺序。

后续训练时，预处理参数从哪里来

当前四行只是教学示例，还没有划分训练与测试集。以后进行真实训练评估时，填补均值等统计参数只从训练集计算，验证和测试使用这些已确定的参数。

原因：未见数据不应参与确定训练流程，也应按同一套转换规则处理。独热编码的列名与顺序同样需要保持一致，这样模型的每个输入位置才有固定含义。

完整流程：对照原理再运行一次

下面程序在项目根目录运行，读取本节配套 CSV。它将整个主流程放在一起，方便离线阅读时查对。

PYTHON

```
import numpy as np
import pandas as pd
import torch

data = pd.read_csv("datasets/ch02/house_tiny.csv")
inputs = data.iloc[:, :2].copy()
targets = data.iloc[:, 2].copy()

mean_rooms = inputs["NumRooms"].mean()
inputs["NumRooms"] = inputs["NumRooms"].fillna(mean_rooms)
encoded = pd.get_dummies(
    inputs, columns=["Alley"],
    dummy_na=True, dtype=np.float32,
)

X = torch.tensor(
    encoded.to_numpy(dtype=np.float32),
    dtype=torch.float32,
)
y = torch.tensor(
    targets.to_numpy(dtype=np.float32),
    dtype=torch.float32,
).reshape(-1, 1)

print(encoded)
print(X.shape, y.shape)
print(torch.isnan(X).any(), torch.isnan(y).any())
```

自检：X 为四行三列，y 为四行一列，均为 float32，不含 NaN。输入和标签保持同一行顺序。程序能运行只是第一步，还需解释每个转换的原因。

综合练习：更多行和列的原始数据

下面是配套的完整人工数据，可以直接在纸上分析缺失数量，也可以运行 CSV 练习。NA 表示缺失。

NumRooms	Area	Alley	Garage	Price
2	60	Pave	NA	106000
NA	80	Gravel	NA	127500
4	NA	Pave	NA	178100
3	90	NA	Yes	140000
NA	70	Gravel	NA	150000
5	100	NA	NA	200000

一次完成以下处理：

1. 取出 Price 作为标签，其余列作为输入。
2. 计算输入各列的缺失数量，找出并删除缺失最多的列。
3. 数值列按各自已知数值的均值填补，不硬编码均值。
4. 对无序类别独热编码，同时表示缺失状态。
5. 创建 float32 的 X 与 (N,1) 形式的 y，检查行数和 NaN。

注意练习表的输入列数多于示例，不能只复制“取前两列”而忽略其他输入。配套文件：`exercises/ch02/02_data_preprocessing.py`。

集中记录：删除了哪列、填补均值、编码后的列名、X/y 的形状，以及下面三个解释。

- A：未知房间数为什么不能默认填 0？均值填补做了什么假设？
- B：把无序类别直接作为 0、1、2 的数值输入，会引入什么关系？独热编码如何表达？
- C：训练集与测试集应各自计算填补均值吗？同一特征的编码列顺序为什么要一致？

阅读后的速查表

环节	方法与关注点
读取文件	read_csv；确认列名、行数和 dtype
分开输入与目标	iloc 或按列名选择；Price 不放入输入
识别缺失	isna().sum()；缺失不等于零
数值填补	mean + fillna；按列计算
删除数据	dropna 删除行；drop(columns=...) 删除列
编码类别	get_dummies；保留缺失状态和特征列名
转成张量	to_numpy(dtype=...) + torch.tensor
检查计算约定	X 行数与 y 行数一致；标签形状对齐

配套完整演示：`lessons/ch02/02_data_preprocessing.py`。下一节为 2.3 线性代数。

资料说明：教材作者为 Aston Zhang、Zachary C. Lipton、Mu Li、Alexander J. Smola；页码基于用户提供的中文 PDF。文中相对路径均从“李沐-深度学习”项目根目录计算。已验证环境：Python 3.8.20、PyTorch 2.4.1+cu121、NumPy 1.23.5、pandas 2.0.3。整理日期：2026-10-15。